

Development of an AI-Based Phishing Detection Tool

- A Practical Application of Machine Learning in
Cybersecurity

Submitted by:
Prakhar Pandey

GitHub Repository:

<https://github.com/PSquare3027/Development-of-an-AI-Based-Phishing-Detection-Tool.git>

1. Abstract

Phishing attacks are one of the most common cybersecurity threats and are often used to trick users into revealing sensitive information. The goal of this project was to build a machine learning system capable of identifying suspicious messages automatically.

The project was developed in Python using the SMS Spam Collection Dataset. Messages were converted into numerical features using TF-IDF vectorization and classified using a Logistic Regression model. The final model achieved an accuracy of 96.32%.

A simple graphical interface was also developed to allow users to enter messages and receive predictions along with confidence scores. This project demonstrates how machine learning can be applied to solve practical cybersecurity problems.

2. Introduction

Phishing is a form of social engineering in which attackers attempt to trick users into revealing passwords, banking information, or other sensitive data. As the number of online messages continues to increase, manually identifying phishing attempts becomes difficult.

This project explores the use of machine learning to automatically classify messages as either legitimate or suspicious. The system was trained using real-world message data and provides instant predictions through a simple user interface.

3. Objectives

- Build a machine learning model capable of detecting suspicious messages.
- Learn how text data can be processed and analyzed.
- Apply machine learning techniques to a cybersecurity problem.

- Develop a simple interface for user interaction.
- Evaluate the effectiveness of the model using real-world data.

4. Dataset and Preprocessing

The SMS Spam Collection Dataset was used for training and testing. The dataset contains 5,572 labeled messages categorized as either legitimate (ham) or spam.

Only the message content and label columns were used. Unnecessary columns were removed before training.

```
1 import pandas as pd
2
3 data = pd.read_csv(
4     r"D:\Code Stuff\Python\Project2\kagglehub\datasets\uciml\sms-spam-collection-dataset\versions\1\spam.csv",
5     encoding="latin-1"
6 )
7
8 print(data.head())
9 print(data.columns)
10 print(data.shape)
11
12
13
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
(.venv) PS D:\Code Stuff\Python> d;; cd 'd:\Code Stuff\Python'; & 'd:\Code Stuff\Python\.venv\Scripts\python.exe' 'c:\Users\Admin\venv\6.6.0-win32-x64\bundled\libs\debugpy\launcher' '61635' '--' 'd:\Code Stuff\Python\Project2\start.py'
v1 v2 Unnamed: 2 Unnamed: 3 Unnamed: 4
0 ham Go until jurong point, crazy.. Available only ... NaN NaN NaN
1 ham Ok lar... Joking wif u oni... NaN NaN NaN
2 spam Free entry in 2 a wkly comp to win FA Cup fina... NaN NaN NaN
3 ham U dun say so early hor... U c already then say... NaN NaN NaN
4 ham Nah I don't think he goes to usf, he lives aro... NaN NaN NaN
Index(['v1', 'v2', 'Unnamed: 2', 'Unnamed: 3', 'Unnamed: 4'], dtype='str')
(5572, 5)
(.venv) PS D:\Code Stuff\Python>
```

Figure 1: Sample records from the dataset after preprocessing.

5. Methodology

The project follows the workflow shown below:

Message

↓

TF-IDF Vectorization

↓

Logistic Regression Model

↓

Prediction

↓

Risk Assessment

TF-IDF vectorization was used to convert text messages into numerical features. A Logistic Regression classifier was then trained using an 80-20 train-test split.

```
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from sklearn.feature_extraction.text import TfidfVectorizer
4 from sklearn.linear_model import LogisticRegression
5 from sklearn.metrics import accuracy_score
6
7 data = pd.read_csv(r"D:\Code Stuff\Python\Project2\kagglehub\datasets\uciml\sms-spam-collection-dataset\versions\1\spam.csv", encoding="latin-1")
8 data = data[["v1", "v2"]]
9 data.columns = ["label", "message"]
10
11 x = data["message"]
12 y = data["label"]
13
14 vectorizer = TfidfVectorizer()
15
16 x = vectorizer.fit_transform(x)
17 X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
18
19 model = LogisticRegression()
20 model.fit(X_train, y_train)
21
22 predictions = model.predict(X_test)
23 accuracy = accuracy_score(y_test, predictions)
24
25 print(f"Accuracy: {accuracy * 100:.2f}%")
26
27
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console + -

```
(.venv) PS D:\Code Stuff\Python> ^C
(.venv) PS D:\Code Stuff\Python>
(.venv) PS D:\Code Stuff\Python> d; cd 'd:\Code Stuff\Python'; & 'd:\Code Stuff\Python\.venv\Scripts\python.exe' 'c:\Users\Admin\.vscode\extensions\ms-python.debug
6.6.0-win32-x64\bundled\libs\debugpy\launcher' '62453' '--' 'd:\Code Stuff\Python\Project2\start.py'
Accuracy: 96.32%
(.venv) PS D:\Code Stuff\Python>
```

Figure 2: Model training and feature extraction process.

6. Results

After training, the model was evaluated using unseen test data.

Model Accuracy: 96.32%

```
25
26 predict = model.predict(X_test)
27 accuracy = accuracy_score(y_test,predict)
28
29 print(f"Accuracy: {accuracy * 100:.2f}%")
30
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Accuracy: 96.32%
(.venv) PS D:\Code Stuff\Python> █

Figure 3: Model accuracy on the testing dataset.

7. User Interface

A simple graphical user interface was developed using Tkinter. Users can enter a message and receive a prediction along with confidence scores.

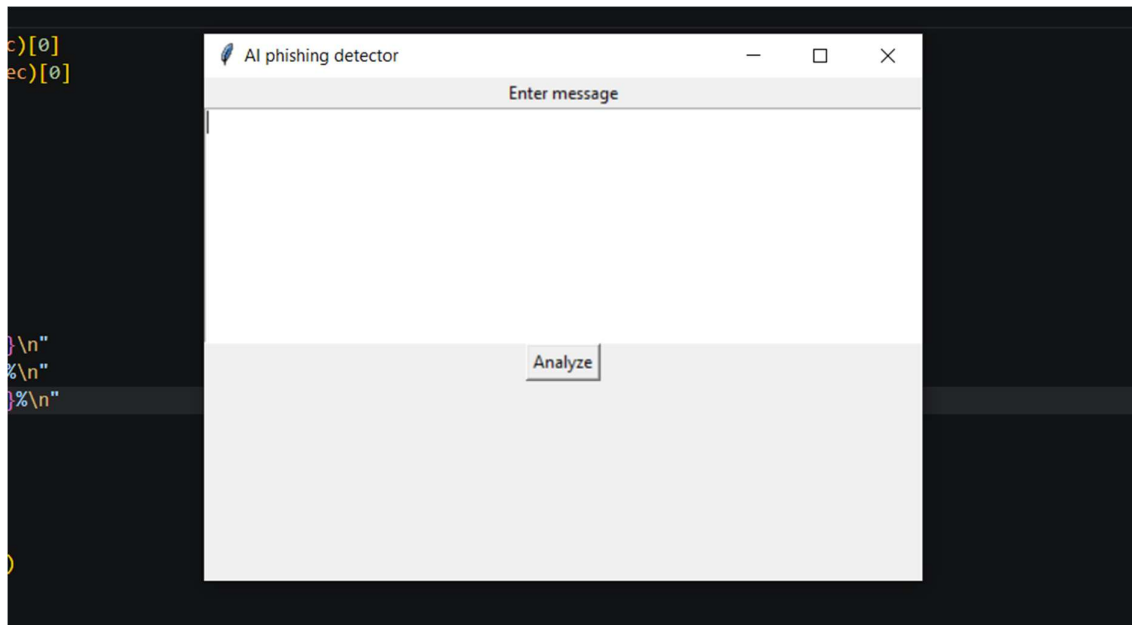


Figure 4: User interface of the phishing detection system.

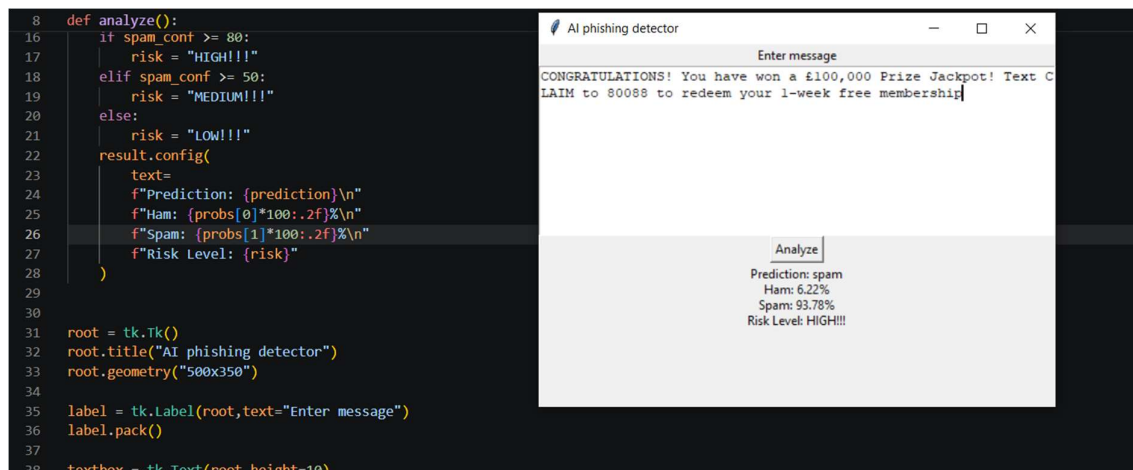


Figure 5: Example phishing message analysis.

8. Limitations

- The model was trained primarily on SMS messages.
- Some messages may still be incorrectly classified.
- Links, attachments, and sender information are not analyzed.
- The project is intended for educational purposes.

9. Future Improvements

- Support for full email phishing detection instead of SMS-only messages.
- Integration with Python's imaplib module to automatically retrieve emails from a user's inbox and analyze them programmatically.
- Deep learning based classifiers such as neural networks for improved detection accuracy.
- URL and attachment analysis to detect malicious links and files.
- Browser extension integration for real-time phishing warnings.
- Multi-language support for detecting phishing attempts in different languages.
- Real-time monitoring and automated threat reporting.

10. Conclusion

This project successfully demonstrated the use of machine learning for phishing detection. Using TF-IDF vectorization and Logistic Regression, the model achieved an accuracy of 96.32% while maintaining a simple and lightweight design.

The project provided practical experience with machine learning, text processing, and cybersecurity concepts. It also highlights how AI can assist users in identifying potentially malicious messages.